

Notebook

February 13, 2025

1 Theoretische und praktische Anwendung der bedingten Wahrscheinlichkeiten

1.1 Aufgabe 2:

Außerirdische können freundlich oder nicht freundlich sein; 75 % sind freundlich. Freundliche Aliens kommen in 90 % der Fälle tagsüber an und 10% nachts, während unfreundliche Aliens immer nachts ankommen. Simulieren Sie wieder wie bei Aufgabe 1 mit Hilfe eines Programms und lösen die Aufgaben zusätzlich mit Bayes Satz.

1.1.1 Situation:

Aus der Aufgabenstellung heraus lassen sich verschiedene Wahrscheinlichkeiten ablesen. Dazu zählen:

- **Wahrscheinlichkeit, dass ein Alien freundlich ist:** $P(F) = 75\%$
- **Wahrscheinlichkeit, dass ein Alien unfreundlich ist:** $P(\neg F) = 25\%$
- **Wahrscheinlichkeit, dass, wenn ein Alien freundlich ist, es tagsüber ankommt:**
 $P(D | F) = 90\%$
- **Wahrscheinlichkeit, dass, wenn ein Alien freundlich ist, es nachts ankommt:**
 $P(\neg D | F) = 10\%$
- **Wahrscheinlichkeit, dass, wenn ein Alien unfreundlich ist, es tagsüber ankommt:**
 $P(D | \neg F) = 0\%$
- **Wahrscheinlichkeit, dass, wenn ein Alien unfreundlich ist, es nachts ankommt:**
 $P(\neg D | \neg F) = 100\%$

1.1.2 Aufgabenteil a)

Wenn ein Außerirdischer nachts kommt, wie wahrscheinlich ist es, dass er freundlich ist? Gesucht ist $P(F | \neg D)$

Mathematische Lösung: Unter der Verwendung des Satz von Bayes kann die bedingte Wahrscheinlichkeit für $P(F | \neg D)$ berechnet werden, wenn die andere bedingte Wahrscheinlichkeit $P(\neg D | F)$ gegeben ist. Mit dem Satz von Bayes folgt damit:

$$P(F | \neg D) = \frac{P(\neg D | F) \cdot P(F)}{P(\neg D)}$$

wobei $P(D)$ berechnet werden kann, indem man die Gesamtzahl der Aliens ermittelt, die nachts ankommen. Dabei gilt, dass 10% der 75% freundlichen Aliens nachts ankommen und 100% der

25% unfreundlichen Aliens nachts ankommen. Demnach lässt sich $P(D)$ mathematisch berechnen durch:

$$\begin{aligned}P(\neg D) &= P(\neg D \mid F) \cdot P(F) + P(\neg D \mid \neg F) \cdot P(\neg F) \\&= 0.1 \cdot 0.75 + 1 \cdot 0.25 \\&= 0.325\end{aligned}$$

Damit folgt mittels Satz von Bayes:

$$\begin{aligned}P(F \mid \neg D) &= \frac{0.1 \cdot 0.75}{0.325} \\&\approx 0.2307\end{aligned}$$

Womit die Wahrscheinlichkeit für das auftreten des Ereignisses “Wenn ein Alien nachts ankommt, dann ist es freundlich” bei 23% liegt.

Programmetechnische Lösung: Auch hier werden zu Beginn wieder die Bibliotheken `random` zur Generierung von Zufallsvariablen und `math` zum späteren Abrunden importiert.

```
[544]: import random
import math
```

Um ein sichereres Ergebnis zu bekommen, werden die Simulationen fünf mal hintereinander durchgeführt und zum Schluss ein Durchschnitt über alle Simulationsabläufe genommen.

```
[545]: loops : int = 5
iterations : int = 10000
```

Die relevanten Ergebnisse zum Lösen der Problemstellung sind die Vorkommen von Aliens bei Nacht und die Vorkommen von freundlichen Aliens bei Nacht. Die Wahrscheinlichkeiten der Simulationsabläufe werden dabei nach jeder Simulation in eine Liste gespeichert und später verarbeitet.

Für jede Iteration innerhalb der der Simulation wird ein freundliches oder unfreundliches Alien generiert. Bei einem Wert unter 0.75, also 75%, wird dieses Alien als freundlich markiert. Ansonsten wird es unfreundlich markiert und zugleich auch als ein Vorkommen bei Nacht markiert, da unfreundliche Aliens nur bei Nacht ankommen.

Ist das Alien freundlich, so wird ein Ankommen bei Tag oder Nacht generiert. Bei einem Wert unter 0.1, also 10%, ist das freundliche Alien bei Nacht angekommen, ansonsten bei Tag.

Danach wird geprüft, ob es überhaupt nächtliche Vorkommen von Aliens gab. Wenn ja, so wird die Wahrscheinlichkeit für das ankommen bei Nacht, mit der Bedingung, dass es ein freundliches Alien ist, über die Division der nächtlichen freundlichen Vorkommen, durch die Gesamtzahl der nächtlichen Vorkommen geteilt.

```
[546]: probaility_list : list = []
probaility_friendly : float = 0.75
probaility_friendly_night : float = 0.1

for j in range(loops):
```

```

count_night : int = 0
count_friendly_night : int = 0

for _ in range(iterations):
    generate_friendly_unfriendly : float = random.random()

    if generate_friendly_unfriendly < probaility_friendly:
        friendly : bool = True

        generate_encounter_day_night : float = random.random()
        if generate_encounter_day_night < probaility_friendly_night:
            arrival : str = "night"

        else:
            arrival : str = "day"

    else:
        friendly : bool = False
        arrival : str = "night"

    if arrival == "night":
        count_night += 1
        if friendly:
            count_friendly_night += 1

    if count_night > 0:
        probaility : float = math.floor((count_friendly_night /
↪count_night) * 10000) / 100
    else:
        probaility : float = 0

    probaility_list.append(probaility)
    print(f"Loop {j + 1}: Die Wahrscheinlichkeit, dass, wenn ein Alien nachts,
↪kommt, es freundlich ist, liegt bei {probaility}%")

```

Loop 1: Die Wahrscheinlichkeit, dass, wenn ein Alien nachts kommt, es freundlich ist, liegt bei 23.0%

Loop 2: Die Wahrscheinlichkeit, dass, wenn ein Alien nachts kommt, es freundlich ist, liegt bei 22.95%

Loop 3: Die Wahrscheinlichkeit, dass, wenn ein Alien nachts kommt, es freundlich ist, liegt bei 23.01%

Loop 4: Die Wahrscheinlichkeit, dass, wenn ein Alien nachts kommt, es freundlich ist, liegt bei 23.95%

Loop 5: Die Wahrscheinlichkeit, dass, wenn ein Alien nachts kommt, es freundlich ist, liegt bei 23.15%

Zuletzt wird dann noch der Durchschnitt über das arithmetische Mittel der Wahrscheinlichkeiten genommen und ausgegeben.

```
[547]: avg_percentage = 0

for p in probaility_list:
    avg_percentage += p

avg_percentage = math.floor(avg_percentage / len(probaility_list) * 100) / 100

print(f"Simulationendurchschnitt: {avg_percentage}%")
```

Simulationendurchschnitt: 23.21%

Das Ergebniss zeigt einen ähnlichen Wert wie bei der mathematischen Herleitung über den Satz von Bayes mit circa. 23%

1.1.3 Aufgabenteil b)

Die Hälfte aller Monster lebt auf Dachböden, der Rest in Kellern. Während 80 % aller Monster weiß sind, sind alle Monster, die auf dem Dachboden leben, weiß. Wie groß ist die Wahrscheinlichkeit, dass ein Monster, das im Keller lebt, weiß ist?

1.1.4 Situation

Aus der angegebenen Aufgabenstellung lassen sich die folgenden Wahrscheinlichkeiten ableiten:

- **Wahrscheinlichkeit, dass ein Monster auf dem Dachboden lebt:** $P(D) = 50\%$
- **Wahrscheinlichkeit, dass ein Monster im Keller lebt:** $P(K) = 50\%$
- **Wahrscheinlichkeit, dass ein Monster weiß ist:** $P(W) = 80\%$
- **Wahrscheinlichkeit, dass, wenn ein Monster im Dachboden lebt, es weiß ist:** $P(W | D) = 100\%$

Mathematische Lösung: Um diese Aufgabe mathematisch zu lösen, kann die Formel für $P(W)$ hergeleitet werden. Diese besagt, dass sich die Wahrscheinlichkeit, dass ein Monster weiß ist, in dem die Gesamtzahl der weißen Monster aus dem Dachböden und Kellern zusammenaddiert werden. Dabei gilt, dass 100% der 50% an Monstern, welche in Dachböden leben weiß sind und ein unbekannter Wert der 50% aller Monster im Keller weiß sind. Daraus ergibt sich die folgende Formel:

$$P(W) = P(W | K) \cdot P(K) + P(W | D) \cdot P(D)$$

Umgestellt auf $P(W | K)$ ergibt sich:

$$\begin{aligned} P(W | K) &= \frac{P(W) - P(W | D) \cdot P(D)}{P(K)} \\ &= \frac{0.8 - 1 \cdot 0.5}{0.5} \\ &= 0.6 \end{aligned}$$

womit die Wahrscheinlichkeit, dass ein Monster, welches im Keller lebt, weiß ist, bei 60% liegt.

Programmtechnische Lösung: Im folgenden wird ein Programmablauf erklärt, welcher die Wahrscheinlichkeit für das Auftreten eines weißen Monster im Keller über Kalibrierung bestimmt wird. Dies geschieht über das Iterieren von Simulationen und darauffolgendes Überprüfen der Wahrscheinlichkeit $P(W) = 80$. Dazu wird der Parameter p eingeführt, welcher die Wahrscheinlichkeit $P(W | K)$ darstellt.

Die Funktion `simulate` ist dabei so aufgebaut, dass sie über die Anzahl der Iterationen Monster generiert die entweder im Keller oder auf dem Dachboden wohnen. Wenn ein Monster im Dachboden wohnt, ist es automatisch weiß, wenn es im Keller wohnt, wird eine Zufallszahl generiert die das Monster weiß oder schwarz machen. Hier kommt die Variable p zum Einsatz, welche zu Beginn des Programmes definiert werden wird. Ist das Monster allgemein weiß, so wird der Zähler der weißen Monster erhöht.

Als Rückgabewert wird die prozentuale Verteilung der weißen Monster auf die Anzahl der Iterationen zurückgegeben.

```
[548]: def simulate(p, iterations):
        count_white : int = 0
        probaility_basement : float = 0.5

        for _ in range(iterations):
            generate_basement_attic : float = random.random()

            if generate_basement_attic < probaility_basement:
                # location : str = "basement"

                generate_black_white = random.random()

                if generate_black_white < p:
                    color : str = "white"

                else:
                    color : str = "black"

            else:
                # location : str = "attic"
                color : str = "white"

            if color == "white":
                count_white += 1

        return count_white / iterations
```

Zu Beginn des Programms werden einige Variablen definiert. Dabei beschreibt die Variable `target_value_white` den Zielrückgabewert der Funktion.

Die Variable `tolerance` beschreibt, wie stark der Wert der Abweichung zwischen Ziel- und Istwert sein muss, damit das Programm vorzeitig endet.

Das `lower_limit` und `upper_limit` beschreiben den Bereich in der sich die Wahrscheinlichkeit p

bewegt und wird je nach Ergebniss der Simulation neu definiert.

```
[549]: target_value_white : float = 0.8
      tolerance : float = 0.001
      lower_limit : float = 0
      upper_limit : float = 1
      iterations : int = 10000
      loops : int = 100
```

Nun iterieren wir und definieren die Startwahrscheinlichkeit des Auftreten eines weißen Monsters im Keller als 50%. Dieser Wert ist nicht richtig, aber wird durch mehrfache Abläufe der Simulation angepasst werden.

Diese Wahrscheinlichkeit wird der Simulation übergeben und die Wahrscheinlichkeit des Auftretens eines weißen Monsters im allgemeinen, pro Simulation abgespeichert.

Dann wird überprüft, ob der Wahrscheinlichkeitswert nahe dem Zielwert von 80% liegt, wenn ja wird das Programm beendet und die Wahrscheinlichkeit für ein weißes Monster im Keller ausgegeben. Wenn nein, so wird überprüft, ob der Wert über, oder unter dem Zielwert liegt und das `upper_limit`, bzw- `lower_limit` auf den aktuellen p-Wert gesetzt, sodass bei der nächsten Iteration ein höherer, bzw. niedrigerer Wert angenommen wird.

```
[550]: for _ in range(loops):
      probaility_basement_white : float = ((lower_limit + upper_limit) / 2)
      probaility_white : int = simulate(probaility_basement_white, iterations)

      difference = abs(probaility_white - target_value_white)

      if difference < tolerance:
          break

      if probaility_white > target_value_white:
          upper_limit = probaility_basement_white

      else:
          lower_limit = probaility_basement_white

      print(f"Die Wahrscheinlichkeit, dass ein Monster, welches im Keller lebt weiß ist, beträgt {math.floor(probaility_basement_white * 10000) / 100}%")
```

Die Wahrscheinlichkeit, dass ein Monster, welches im Keller lebt weiß ist, beträgt 60.09%

Das Ergebniss ähnelt hier bei wieder stark der mathematischen Lösung von ca. 60%

This notebook was converted with `convert.ploomber.io`